UNIVERSIDAD NACIONAL DE INGENIERÍA
Facultad de Ingeniería Industrial y de Sistemas
UNIDAD DE POSGRADO

Práctica Dirigida N.° 2 — Backpropagation Manual y CNN para MNIST

Modalidad de trabajo
Práctica individual o en parejas, para desarrollarse durante la sesión de clase con computadora (Google Colab recomendado). Duración total: 1 hora 15 minutos (Parte 1: 30 min · Parte 2: 45 min).

I. Objetivos de la práctica

- Reforzar el algoritmo de backpropagation calculando a mano el gradiente de un grafo computacional pequeño, aplicando la regla de la cadena nodo por nodo.
- Implementar y entrenar una red neuronal convolucional (CNN) simple para clasificar dígitos de MNIST usando PyTorch.
- Comparar el desempeño de la CNN frente al MLP construido en la Práctica Dirigida N.° 1.

II. Parte 1 — Backpropagation manual sobre un grafo computacional (30 min)

En clase revisamos el siguiente ejemplo (circuito de una neurona sigmoide, calculado como referencia), donde cada nodo del grafo muestra en verde el valor calculado durante el forward pass, y en rojo el gradiente calculado durante el backward pass usando la regla [gradiente local] × [gradiente aguas arriba]:

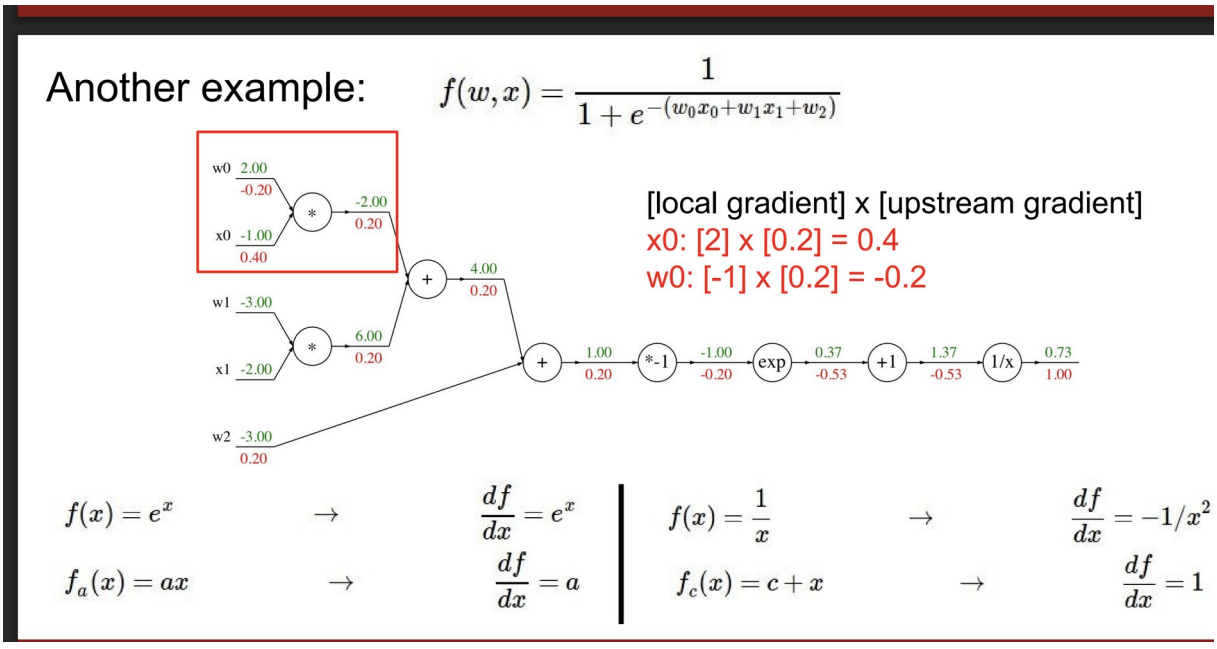


Figura de referencia — circuito de una neurona sigmoide (fuente: CS231n, Stanford)

Ejercicio — mismo circuito, valores nuevos

La función es la misma: una neurona sigmoide

$$f(w, x) = 1 / (1 + e^{-(w_0x_0 + w_1x_1 + w_2)})$$

con la misma secuencia de nodos que en la figura (multiplicar → sumar → sumar → multiplicar por −1 → exponencial → sumar 1 → invertir), pero con estos valores de entrada:

w_0	x_0	w_1	x_1	w_2
1.00	-2.00	-2.00	-1.00	-1.00

1. Calculen el forward pass: completen el valor de salida de cada nodo del circuito (verde), en el mismo orden que la figura de referencia (producto 1, producto 2, suma 1, suma 2, ×(−1), exp, +1, 1/x). El resultado final f(w,x) debe quedar entre 0 y 1.
2. Calculen el backward pass nodo por nodo, de derecha a izquierda: para cada nodo escriban (a) la fórmula del gradiente local, (b) su valor numérico evaluado en el resultado del forward pass, y (c) el gradiente que sale del nodo (local × upstream). Usen como ayuda las derivadas ya dadas en la figura de referencia: $d/dx[1/x] = -1/x^2$, $d/dx[e^x] = e^x$, $d/dx[c+x] = 1$, $d/dx[ax] = a$.
3. A partir del backward pass, reporten los 5 gradientes finales: $\partial f/\partial w_0$, $\partial f/\partial x_0$, $\partial f/\partial w_1$, $\partial f/\partial x_1$ y $\partial f/\partial w_2$.
4. (Verificación rápida) En una celda de Python, calculen f(w,x) con estos valores y comparen con su resultado del forward pass a mano. Si tienen tiempo, verifiquen también los gradientes con torch.autograd (requires_grad=True y .backward()) sobre las mismas variables.

III. Parte 2 — CNN simple para MNIST (45 min)

Ahora construimos una red neuronal convolucional (CNN) para el mismo problema de clasificación de dígitos MNIST que trabajamos con el MLP en la Práctica Dirigida N.° 1. Reutilicen el mismo bloque de carga de datos (transforms, MNIST, DataLoader) de esa práctica; solo cambia la arquitectura del modelo.

Arquitectura a implementar

Capa	Especificación
Conv1	nn.Conv2d(1, 16, kernel_size=3, padding=1) → ReLU → nn.MaxPool2d(2)
Conv2	nn.Conv2d(16, 32, kernel_size=3, padding=1) → ReLU → nn.MaxPool2d(2)
Flatten	aplanar el tensor resultante
FC1	nn.Linear(?, 128) → ReLU
FC2	nn.Linear(128, 10)

Completen el código siguiente (los espacios marcados con ___ requieren que ustedes calculen o completen el valor: canales de entrada/salida de conv2, tamaño de entrada de fc1, y las dimensiones tras el segundo MaxPool2d):

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(___, ___, kernel_size=3, padding=1)
        self.pool  = nn.MaxPool2d(2)
        self.relu  = nn.ReLU()
        self.fc1   = nn.Linear(___, 128)
        self.fc2   = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))  # 28x28 -> 14x14
        x = self.pool(self.relu(self.conv2(x)))  # 14x14 -> ___x___
        x = x.view(x.size(0), -1)                # flatten
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN().to(DEVICE)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

Tareas a realizar

1. Calcular a mano las dimensiones espaciales que quedan tras cada MaxPool2d(2) (28→14→___) y completar los espacios en blanco del código (canales de conv2 y tamaño de entrada de fc1).
2. Reutilizar las funciones train_one_epoch() y evaluate() de la Práctica Dirigida N.° 1 (o escribir un ciclo equivalente) para entrenar la CNN durante 3 épocas sobre el mismo train/val/test split.
3. Registrar el Test Accuracy final de la CNN y compararlo con el Test Accuracy del MLP obtenido en la práctica anterior.
4. Contar cuántos parámetros entrenables tiene la CNN (sum(p.numel() for p in model.parameters())) y compararlo contra los parámetros del MLP. ¿Cuál modelo tiene más parámetros? ¿Cuál obtuvo mejor accuracy?

5. (Opcional, si sobra tiempo) Agregar nn.BatchNorm2d después de cada convolución y observar si cambia la velocidad de convergencia en las primeras 3 épocas.

Pregunta de cierre

En 2-3 líneas: ¿por qué una CNN suele necesitar menos parámetros que un MLP para lograr un accuracy igual o mejor en imágenes como las de MNIST? Relacionen su respuesta con el concepto de compartición de pesos (weight sharing) de la convolución.